



CWE-807: Reliance on Untrusted Inputs in a Security Decision

Weakness ID: 807

Vulnerability Mapping: ALLOWED**Abstraction:** Base

View customized information:

Conceptual

Operational

Mapping
Friendly**Complete**

Custom

▼ Description

The product uses a protection mechanism that relies on the existence or values of an input, but the input can be modified by an untrusted actor in a way that bypasses the protection mechanism.

▼ Extended Description

Developers may assume that inputs such as cookies, environment variables, and hidden form fields cannot be modified. However, an attacker could change these inputs using customized clients or other attacks. This change might not be detected. When security decisions such as authentication and authorization are made based on the values of these inputs, attackers can bypass the security of the software.

Without sufficient encryption, integrity checking, or other mechanism, any input that originates from an outsider cannot be trusted.

▼ Common Consequences

Impact	Details
Bypass Protection Mechanism; Gain Privileges or Assume Identity; Varies by Context	Scope: Confidentiality, Access Control, Availability, Other Attackers can bypass the security decision to access whatever is being protected. The consequences will depend on the associated functionality, but they can range from granting additional privileges to untrusted users to bypassing important security checks. Ultimately, this weakness may lead to exposure or modification of sensitive data, system crash, or execution of arbitrary code.

▼ Potential Mitigations

Phase(s)	Mitigation
Architecture and Design	Strategy: Attack Surface Reduction Store state information and sensitive data on the server side only. Ensure that the system definitively and unambiguously keeps track of its own state and user state and has rules defined for legitimate state transitions. Do not allow any application user to affect state directly in any way other than through legitimate actions leading to state transitions. If information must be stored on the client, do not do so without encryption and integrity checking, or otherwise having a mechanism on the server side to catch tampering. Use a message authentication code (MAC) algorithm, such as Hash Message Authentication Code (HMAC) [REF-529]. Apply this against the state or sensitive data that has to be exposed, which can guarantee the integrity of the data - i.e., that the data has not been modified. Ensure that a strong hash function is used (CWE-328).
Architecture and Design	Strategy: Libraries or Frameworks Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. With a stateless protocol such as HTTP, use a framework that maintains the state for you. Examples include ASP.NET View State [REF-756] and the OWASP ESAPI Session Management feature [REF-45]. Be careful of language features that provide state support, since these might be provided as a convenience to the programmer and may not be considering security.

Architecture and Design	For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.
Operation; Implementation	Strategy: <i>Environment Hardening</i> When using PHP, configure the application so that it does not use <code>register_globals</code>. During implementation, develop the application so that it does not rely on this feature, but be wary of implementing a <code>register_globals</code> emulation that is subject to weaknesses such as CWE-95, CWE-621, and similar issues.
Architecture and Design; Implementation	Strategy: <i>Attack Surface Reduction</i> Understand all the potential areas where untrusted inputs can enter your software: parameters or arguments, cookies, anything read from the network, environment variables, reverse DNS lookups, query results, request headers, URL components, e-mail, files, filenames, databases, and any external systems that provide data to the application. Remember that such inputs may be obtained indirectly through API calls. Identify all inputs that are used for security decisions and determine if you can modify the design so that you do not have to rely on submitted inputs at all. For example, you may be able to keep critical information about the user's session on the server side instead of recording it within external data.

▼ Relationships

📘 ▼ Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	P	693	Protection Mechanism Failure
ParentOf	ⓑ	302	Authentication Bypass by Assumed-Immutable Data
ParentOf	Ⓥ	350	Reliance on Reverse DNS Resolution for a Security-Critical Action
ParentOf	Ⓥ	784	Reliance on Cookies without Validation and Integrity Checking in a Security Decision

▼ Relevant to the view "Software Development" (View-699)

Nature	Type	ID	Name
MemberOf	Ⓢ	1006	Bad Coding Practices

▼ Relevant to the view "Architectural Concepts" (View-1008)

Nature	Type	ID	Name
MemberOf	Ⓢ	1012	Cross Cutting

▼ Modes Of Introduction

Phase	Note
Architecture and Design	COMMISSION: This weakness refers to an incorrect design related to an architectural security tactic.
Implementation	

▼ Applicable Platforms

Languages	Class: Not Language-Specific (<i>Undetermined Prevalence</i>)
Technologies	Class: Not Technology-Specific (<i>Undetermined Prevalence</i>) Class: Web Based (<i>Undetermined Prevalence</i>) Web Server (<i>Undetermined Prevalence</i>)

▼ Likelihood Of Exploit

High

▼ Demonstrative Examples**Example 1**

The following code excerpt reads a value from a browser cookie to determine the role of the user.

*Example Language: Java**(bad code)*

```
Cookie[] cookies = request.getCookies();
for (int i = 0; i < cookies.length; i++) {
    Cookie c = cookies[i];
    if (c.getName().equals("role")) {
        userRole = c.getValue();
    }
}
```

Example 2

The following code could be for a medical records application. It performs authentication by checking if a cookie has been set.

*Example Language: PHP**(bad code)*

```
$auth = $_COOKIES['authenticated'];
if (! $auth) {
    if (AuthenticateUser($_POST['user'], $_POST['password']) == "success") {
        // save the cookie to send out in future responses
        setcookie("authenticated", "1", time()+60*60*2);
    }
    else {
        ShowLoginScreen();
        die("\n");
    }
}
DisplayMedicalHistory($_POST['patient_ID']);
```

The programmer expects that the AuthenticateUser() check will always be applied, and the "authenticated" cookie will only be set when authentication succeeds. The programmer even diligently specifies a 2-hour expiration for the cookie.

However, the attacker can set the "authenticated" cookie to a non-zero value such as 1. As a result, the \$auth variable is 1, and the AuthenticateUser() check is not even performed. The attacker has bypassed the authentication.

Example 3

In the following example, an authentication flag is read from a browser cookie, thus allowing for external control of user state data.

*Example Language: Java**(bad code)*

```
Cookie[] cookies = request.getCookies();
for (int i = 0; i < cookies.length; i++) {
    Cookie c = cookies[i];
    if (c.getName().equals("authenticated") && Boolean.TRUE.equals(c.getValue())) {
        authenticated = true;
    }
}
```

Example 4

The following code samples use a DNS lookup in order to decide whether or not an inbound request is from a trusted host. If an attacker can poison the DNS cache, they can gain trusted status.

Example Language: **C**

(bad code)

```
struct hostent *hp; struct in_addr myaddr;
char* tHost = "trustme.example.com";
myaddr.s_addr = inet_addr(ip_addr_string);

hp = gethostbyaddr((char *) &myaddr, sizeof(struct in_addr), AF_INET);
if (hp && !strcmp(hp->h_name, tHost, sizeof(tHost))) {
    trusted = true;
} else {
    trusted = false;
}
```

Example Language: **Java**

(bad code)

```
String ip = request.getRemoteAddr();
InetAddress addr = InetAddress.getByName(ip);
if (addr.getCanonicalHostName().endsWith("trustme.com")) {
    trusted = true;
}
```

Example Language: **C#**

(bad code)

```
IPAddress hostIPAddress = IPAddress.Parse(RemoteIpAddress);
IPEndPoint hostInfo = Dns.GetHostByAddress(hostIPAddress);
if (hostInfo.HostName.EndsWith("trustme.com")) {
    trusted = true;
}
```

IP addresses are more reliable than DNS names, but they can also be spoofed. Attackers can easily forge the source IP address of the packets they send, but response packets will return to the forged IP address. To see the response packets, the attacker has to sniff the traffic between the victim machine and the forged IP address. In order to accomplish the required sniffing, attackers typically attempt to locate themselves on the same subnet as the victim machine. Attackers may be able to circumvent this requirement by using source routing, but source routing is disabled across much of the Internet today. In summary, IP address verification can be a useful part of an authentication scheme, but it should not be the single factor required for authentication.

▼ Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2009-1549	Attacker can bypass authentication by setting a cookie to a specific value.
CVE-2009-1619	Attacker can bypass authentication and gain admin privileges by setting an "admin" cookie to 1.
CVE-2009-0864	Content management system allows admin privileges by setting a "login" cookie to "OK."
CVE-2008-5784	e-dating application allows admin privileges by setting the admin cookie to 1.
CVE-2008-6291	Web-based email list manager allows attackers to gain admin privileges by setting a login cookie to "admin."

▼ Weakness Ordinalities

Ordinality	Description
Primary	(where the weakness exists independent of other weaknesses)

▼ Detection Methods

Method	Details

Manual Static Analysis	Since this weakness does not typically appear frequently within a single software package, manual white box techniques may be able to provide sufficient code coverage and reduction of false positives if all potentially-vulnerable operations can be assessed within limited time constraints. Effectiveness: High Note:The effectiveness and speed of manual analysis will be reduced if there is not a centralized security mechanism, and the security logic is widely distributed throughout the software.
Automated Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Bytecode Weakness Analysis - including disassembler + source code weakness analysis • Binary Weakness Analysis - including disassembler + source code weakness analysis Effectiveness: SOAR Partial
Manual Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Binary / Bytecode disassembler - then use manual analysis for vulnerabilities & anomalies Effectiveness: SOAR Partial
Dynamic Analysis with Automated Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Web Application Scanner • Web Services Scanner • Database Scanners Effectiveness: SOAR Partial
Dynamic Analysis with Manual Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Fuzz Tester • Framework-based Fuzzer • Monitored Virtual Environment - run potentially malicious code in sandbox / wrapper / virtual machine, see if it does anything suspicious Effectiveness: SOAR Partial
Manual Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Manual Source Code Review (not inspections) Effectiveness: High
Automated Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful:

	Cost effective for partial coverage: • Source code Weakness Analyzer • Context-configured Source Code Weakness Analyzer Effectiveness: SOAR Partial
Architecture or Design Review	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Inspection (IEEE 1028 standard) (can apply to requirements, design, source code, etc.) • Formal Methods / Correct-By-Construction Cost effective for partial coverage: • Attack Modeling Effectiveness: High

▼ Memberships

			
Nature	Type	ID	Name
MemberOf	C	803	2010 Top 25 - Porous Defenses
MemberOf	C	859	The CERT Oracle Secure Coding Standard for Java (2011) Chapter 16 - Platform Security (SEC)
MemberOf	C	866	2011 Top 25 - Porous Defenses
MemberOf	C	878	CERT C++ Secure Coding Section 10 - Environment (ENV)
MemberOf	V	884	CWE Cross-section
MemberOf	C	1348	OWASP Top Ten 2021 Category A04:2021 - Insecure Design
MemberOf	C	1365	ICS Communications: Unreliability
MemberOf	C	1373	ICS Engineering (Construction/Deployment): Trust Model Problems
MemberOf	C	1413	Comprehensive Categorization: Protection Mechanism Failure
MemberOf	C	1441	OWASP Top Ten 2025 Category A06:2025 - Insecure Design

▼ Vulnerability Mapping Notes

Usage	ALLOWED <i>(this CWE ID may be used to map to real-world vulnerabilities)</i>
Reason	Acceptable-Use
Rationale	This CWE entry is at the Base level of abstraction, which is a preferred level of abstraction for mapping to the root causes of vulnerabilities.
Comments	Carefully read both the name and description to ensure that this mapping is an appropriate fit. Do not try to 'force' a mapping to a lower-level Base/Variant simply to comply with this preferred level of abstraction.

▼ Taxonomy Mappings

Mapped Taxonomy Name	Node ID	Fit	Mapped Node Name
The CERT Oracle Secure Coding Standard for Java (2011)	SEC09-J		Do not base security checks on untrusted sources

▼ References

[REF-754]	Frank Kim. "Top 25 Series - Rank 6 - Reliance on Untrusted Inputs in a Security Decision". SANS Software Security Institute. 2010-03-05. < https://www.sans.org/blog/top-25-series-rank-6-reliance-on-untrusted-inputs-in-a-security-decision/ >. (URL validated: 2023-04-07)
[REF-529]	"HMAC". Wikipedia. 2011-08-18. < https://en.wikipedia.org/wiki/HMAC >. (URL validated: 2023-04-07)
[REF-756]	Scott Mitchell. "Understanding ASP.NET View State". Microsoft. 2004-05-15. < https://learn.microsoft.com/en-us/previous-versions/dotnet/articles/ms972976(v=msdn.10)?redirectedfrom=MSDN >. (URL validated: 2023-04-07)
[REF-45]	OWASP. "OWASP Enterprise Security API (ESAPI) Project". < https://owasp.org/www-project-enterprise-security-api/ >. (URL validated: 2025-07-24)
[REF-1479]	Gregory Larsen, E. Kenneth Hong Fong, David A. Wheeler and Rama S. Moorthy. "State-of-the-Art Resources (SOAR) for Software Vulnerability Detection, Test, and Evaluation". 2014-07. < https://www.ida.org/-/media/feature/publications/s/st/stateoftheart-resources-soar-for-software-vulnerability-detection-test-and-evaluation/p-5061.ashx >. (URL validated: 2025-09-05)

▼ Content History

▼ Submissions		
Submission Date	Submitter	Organization
2010-01-18 (CWE 1.8, 2010-02-16)	CWE Content Team	MITRE
► Modifications		